



Report

Artificial Intelligence

Assignment 3

Asra Anees

2023/01/15

Contents

Introduction:.....	3
a. Generate data points in Python using:.....	3
Regression models	3
Linear regression.....	3
b. Establish a linear regression model to predict y data (you can use Sklearn):	4
Polynomial regression.....	4
c. Establish a polynomial model with the highest power of 2 to predict y data (you can use Sklearn).	4
Neural network.....	5
d. Establish a three-layer (including input, 1 hidden (6 nodes), and output layer) neural network to predict y data (Keras or TensorFlow is recommended). Split the dataset into training 80% and testing data 20% using the following code in Python:.....	5
Mean Squared Error	6
e. Calculate and compare mean squared errors of three models in (b), (c), and (d).	6
Plot data	6
f. Plot data points and curve of predictions in (b) and (c).	6
Conclusion	7

Introduction:

A regression model is a statistical method used to predict a continuous value, such as a stock price or temperature, based on historical data. A neural network is a type of machine-learning model inspired by the structure of the human brain. It is composed of layers of interconnected "neurons" that process and transmit information.

The report presents regression models and compares them to neural network models. Generate data points that follow the square function with some random noise added to each data point. The goal is to describe the difference between different regression models and how they compare to a neural network model. Implementation is done in Python. By using python libraries Sklearn, Keras, and/or TensorFlow to find the solution. The report includes information regarding the regression models and the results of all steps *a* to *f*. The code is in the "Assignmet-3" python file.

a. Generate data points in Python using:

```
#creating a data points
x_data = np.linspace(-0.5, 0.5, 200)[: , np.newaxis]
noise = np.random.normal(0, 0.02, x_data.shape)
y_data = np.square(x_data) + noise
```

Regression models

Regression models are used to identify patterns in data and make predictions about future outcomes. They are based on the idea that there is a relationship between a set of input variables (also called features or predictors) and a single output variable (also called the response or target). This model is used in many different fields such as finance, economics, biology, and engineering to make predictions, understand relationships between variables and make decisions. There are several types of regression models, including linear regression and polynomial regression.

Linear regression

Linear regression is the simplest and most widely used regression method, it assumes that the relationship between the input variables and the output variable is linear.

Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data. The equation of a linear regression model can be represented as:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

where y is the dependent variable, $x_1, x_2, \dots, x_n$ are the independent variables, b_0 is the y-intercept also known as the constant term, and $b_1, b_2, \dots, b_n$ are the coefficients of the independent variables.

The y-intercept, represented by b_0 , is the point at which the regression line crosses the y-axis. It represents the expected value of the dependent variable when all independent variables are equal to zero.

It can be thought of as a baseline or starting point for the relationship between the independent and dependent variables.

It's important to note that, the y-intercept is only meaningful when the independent variable(s) is zero, otherwise it doesn't have a physical meaning. In some cases, the y-intercept might not be a meaningful value in the context of the problem being studied, and it can be dropped from the model or set to zero.

The coefficients of the independent variables, represented by $b_1, b_2, \dots, b_n$, are determined by minimizing the sum of the squared differences between the predicted values and the observed values. Once the coefficients have been determined, the linear regression equation can be used to make predictions about the dependent variable given new values of the independent variables.

b. Establish a linear regression model to predict y data (you can use Sklearn):

```
#Create linear regression model
regression = linear_model.LinearRegression()
#train the model
regression.fit(x_data, y_data)
#prediction Making
y_pred = regression.predict(x_data)
```

The LinearRegression class from sklearn.linear_model are uses to create a linear regression model. For fitting this model to the training data using the fit() method, and using it to make predictions on the y data using the predict() method.

Polynomial regression

Polynomial regression allows for the modeling of non-linear relationships between the input variables and the output variable by adding polynomial terms to the linear regression equation.

In polynomial regression, the relationship between the independent variable x and the dependent variable y is modeled as an n^{th} degree polynomial. The equation of a polynomial regression model can be represented as:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

Where y is the dependent variable, x is the independent variable, b_0 is the y -intercept, also known as the constant term $b_1, b_2, \dots, b_n$ are the coefficients of the polynomial terms.

The y -intercept, represented by b_0 , is the point at which the polynomial equation crosses the y -axis. It represents the expected value of the dependent variable when the independent variable x is equal to zero.

The coefficients of the polynomial terms, represented by $b_1, b_2, \dots, b_n$ are determined by minimizing the sum of the squared differences between the predicted values and the observed values. Once the coefficients have been determined, the polynomial regression equation can be used to make predictions about the dependent variable given new values of the independent variable.

It's important to note that the coefficients of polynomial regression are unique for each term of the polynomial equation and they indicate the weight of the corresponding term in the equation, and together with the y -intercept they determine the polynomial equation that best fits the data. The degree of the polynomial equation is determined by the number of polynomial terms in the equation and it determines the complexity of the model. The higher the degree of the polynomial, the more complex the model and the better it will fit the data, but also the more prone to overfitting the data.

c. Establish a polynomial model with the highest power of 2 to predict y data (you can use Sklearn).

```
#polynomial regression
poly = PolynomialFeatures(degree=2)
x_polyData = poly.fit_transform(x_data)
#Create Linear Regression model
poly_reg = linear_model.LinearRegression()
#Train the model
poly_reg.fit(x_polyData, y_data) #Ground truth Ydata
# polynomial Regression y pridiction
y_predPoly= poly_reg.predict(x_polyData)
```

The PolynomialFeatures class from sklearn.preprocessing uses to create a polynomial feature transformer with the highest power of 2. Then using this transformer to transform the x_data by creating new features that are the powers of the original features.

Then, we are using the LinearRegression class from sklearn.linear_model to create a linear regression model, fitting this model to the transformed y_data and using it to make predictions on the transformed new feature (x_polyFt) data.

It's important to note that, polynomial regression can be sensitive to the choice of the degree of the polynomial, and it's a good idea to experiment with different degrees to find the one that gives the best performance. Increasing the degree of the polynomial will increase the model's complexity and make it more prone to overfitting the data.

Neural network

A neural network is a type of machine-learning model inspired by the structure of the human brain. It is composed of layers of interconnected "neurons" that process and transmit information. These neurons are connected by edges called synapses, which are associated with weights.

The basic building block of a neural network is the artificial neuron, which takes in inputs, performs a computation on them, and produces an output. The computation performed by a neuron is called the activation function.

A neural network can have one or more layers, depending on the complexity of the problem. A neural network with one layer is called a single-layer perceptron and it is used for linear binary classification problems, while networks with more than one layer are called multi-layer perceptrons or simply neural networks.

There are several types of neural networks, including feedforward, recurrent, and convolutional neural networks. Feedforward neural networks, also known as multi-layer perceptrons (MLP), are the most common type of neural network. They are composed of an input layer, one or more hidden layers, and an output layer. The input layer receives the input data, the hidden layers process the data, and the output layer produces the predictions.

Neural networks are particularly useful for tasks that involve large amounts of data or nonlinear relationships. They are used in a wide range of applications, such as image and speech recognition, natural language processing, and decision-making.

In comparison, to the regression model, a neural network model is a more complex and flexible model than traditional regression models. Neural networks can model nonlinear relationships, handle multiple input and output variables, and are suitable for a wide range of problems. However, they also require more data, computational resources, and tuning to achieve good performance.

d. Establish a three-layer (including input, 1 hidden (6 nodes), and output layer) neural network to predict y data (Keras or TensorFlow is recommended). Split the dataset into training 80% and testing data 20% using the following code in Python:

```
model = tf.keras.models.Sequential()
#1 input layer and 1 hidden layer with 6 nodes
model.add(tf.keras.layers.Dense(6, input_dim=x_trainFt.shape[1], activation="relu"))
#output Layer
model.add(tf.keras.layers.Dense(1, activation='linear'))
model.compile(optimizer='adam', loss='mean_squared_error')
m_hist = model.fit(x_trainFt, y_trainFt, batch_size = 10, epochs=100, verbose=1, validation_data=(x_testFt, y_testFt))
```

The Sequential model from Keras /TensorFlow use to create a neural network with one input layer, one hidden layer with 6 nodes, and one output layer. The input dimension is defined by the number of features

in the `x_train` data, and the activation function used in the hidden layer is 'relu' and in the output layer is 'linear'. The 'adam' use as an optimizer and 'mean_squared_error' as the loss function. Then, the model is fitted to the training data using the `fit()` method and evaluated on the feature test data using the `evaluate()` method.

Mean Squared Error

The mean squared error (MSE) is a commonly used measure of the difference between the predicted values and the true values in a dataset. It is calculated as the average of the squared differences between the predicted values ($\hat{y}$) and the true values (y) of the dependent variable. The equation for the MSE is:

$$MSE = \frac{1}{n} \sum (\hat{y} - y)^2$$

Where n is the number of observations in the dataset.

e. Calculate and compare mean squared errors of three models in (b), (c), and (d).

In linear regression, the MSE is used to evaluate the performance of the model by measuring the difference between the predicted values and the true values of the dependent variable. The lower the MSE, the better the model is at fitting the data.

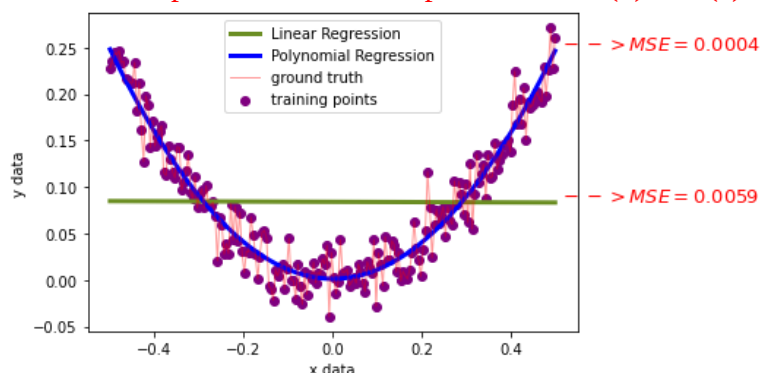
In polynomial regression, the MSE is also used to evaluate the performance of the model by measuring the difference between the predicted values and the true values of the dependent variable. The lower the MSE, the better the model is at fitting the data.

In neural networks, the MSE is commonly used as a loss function to measure the difference between the predicted values and the true values of the dependent variable. The lower the MSE, the better the model is at fitting the data.

```
#linear regression mean squared error
lg_mse = format(mean_squared_error(y_data, y_pred), '.4f')
print("linear regression MSE:", lg_mse)
print("-----\n")
#Polynomial regression mean squared error
poly_mse= format(mean_squared_error(y_data, y_predPoly), '.4f')
print("Polynomial regression MSE:", poly_mse)
print("-----\n")
#Neural Network mean squared error
test_loss = model.evaluate(x_testFt, y_testFt, verbose=0)
print("Neural Network MSE: ", format(test_loss, '.4f'))
```

Plot data

f. Plot data points and curve of predictions in (b) and (c).



Conclusion

linear and polynomial regression models are simple, fast, and suitable for problems with linear or nonlinear relationships between variables, respectively. Neural network models are more complex and flexible than traditional regression.